```
prog.rb:3:in `block (2 levels) in <main>': Boom! (RuntimeError)
Waiting
0
2
3
Done
```

You normally don't sleep waiting for threads to terminate—you'd use join. If you join to a thread that has raised an exception, then that exception will be raised in the thread that does the joining:

```
threads = 4.times.map do |number|
  Thread.new(number) do |i|
    raise "Boom!" if i == 1
    print "#{i}\n"
  end
end

puts "Waiting"
threads.each do |t|
  begin
    t.join
  rescue RuntimeError => e
    puts "Failed: #{e.message}"
  end
end
puts "Done"
```

produces:

```
#<Thread:0x00000001042e8150 prog.rb:2 run> terminated with exception
(report_on_exception is true):
prog.rb:3:in `block (2 levels) in <main>': Boom! (RuntimeError)
Waiting
0
2
Failed: Boom!
3
Done
```

However, set abort_on_exception to true or use -d to turn on the debug flag, and an unhandled exception kills the main thread, so the message Done never appears.

```
Thread.abort_on_exception = true
threads = 4.times.map do |number|
  Thread.new(number) do |i|
    raise "Boom!" if i == 1
    print "#{i}\n"
  end
end
puts "Waiting"
threads.each {|t| t.join }
puts "Done"
```

produces:

```
#<Thread:0x00000001041f60f8 prog.rb:3 run> terminated with exception
(report_on_exception is true):
```